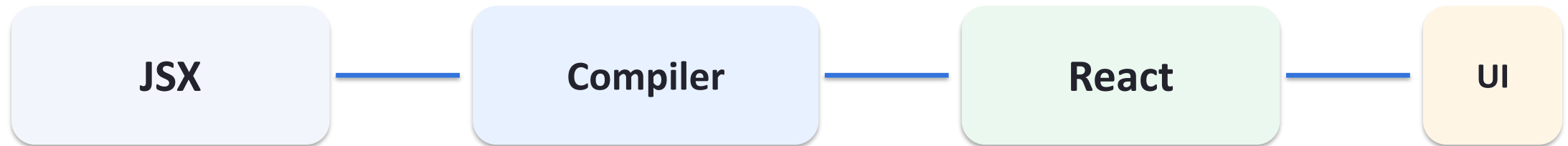


React + JSX: From Code to UI

A simple picture of what happens after you write `<div>Hello</div>`



1. What is JSX?

JSX lets us write HTML-like syntax inside JavaScript.

```
const App = () => <div>Hello  
  World</div>;
```

**Think of JSX as a convenient
way to describe the UI.**

JavaScript

JSX

HTML-like UI description

2. How JSX becomes JavaScript

Browsers do not directly execute JSX. Your build tool/compiler transforms it.

```
<div>Hello World</div>
```

```
jsx("div", {\n  children:  
    "Hello World"\n})
```

Modern JSX Transform: the compiler can use react/jsx-runtime automatically.

Older React: JSX was commonly compiled to `React.createElement(...)`

3. So, do I still write import React?

For basic JSX, usually no. Import what you actually use.

 Usually not needed for JSX

```
const App = () =>\n  <h1>Hello</h1>;
```

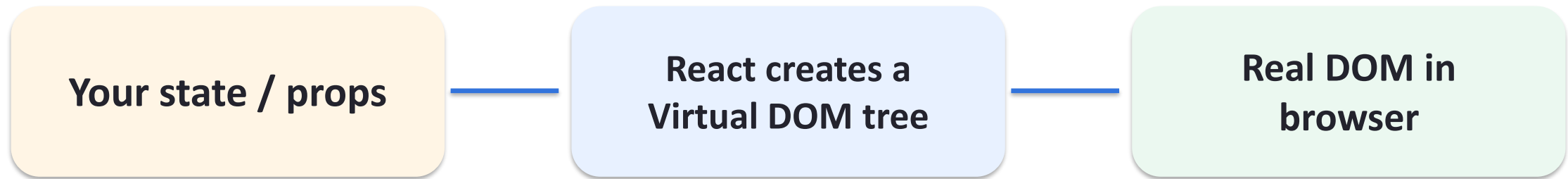
 Import hooks you use

```
import { useState } from  
"react";\n\nconst App = () => ...
```

If you explicitly write `React.createElement`, then you need a React reference.

4. What is the Virtual DOM?

A simple mental model: React keeps a lightweight description of the UI.

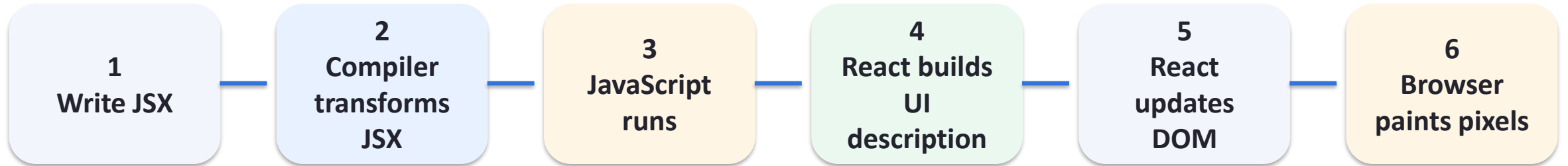


When data changes, React compares the new UI description with the previous one and updates the necessary DOM parts.

Goal: avoid unnecessary DOM work and make UI updates easier to manage.

5. Complete journey: code → screen

This is the big picture to remember.



JSX is developer-friendly syntax. The browser ultimately receives normal JavaScript and DOM updates.

Quick recap

Five points students should remember.

JSX

HTML-like syntax written inside JavaScript.

Compiler

Transforms JSX into JavaScript the runtime can use.

React 17+ JSX Transform

Basic JSX does not require import React.

Virtual DOM

A lightweight UI representation React compares between renders.

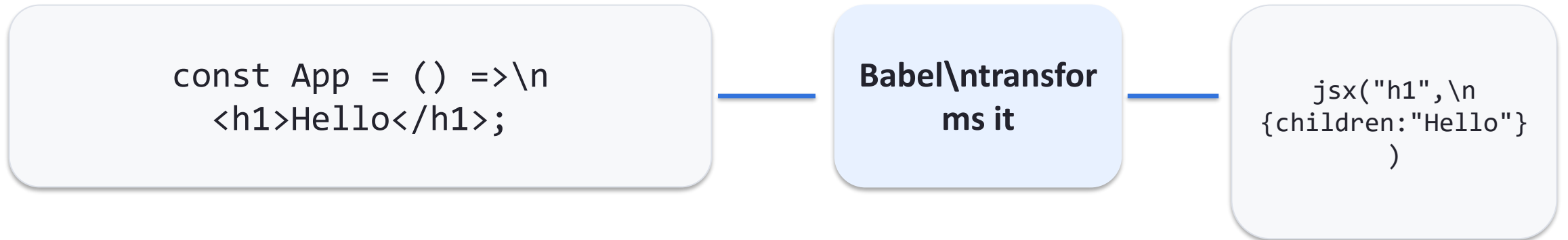
Real DOM

React applies the needed changes to the browser DOM.

Tip: Don't memorize the compiler output — understand the flow.

6. What is Babel?

Babel is a JavaScript compiler/transpiler often used to transform JSX and newer JavaScript syntax.

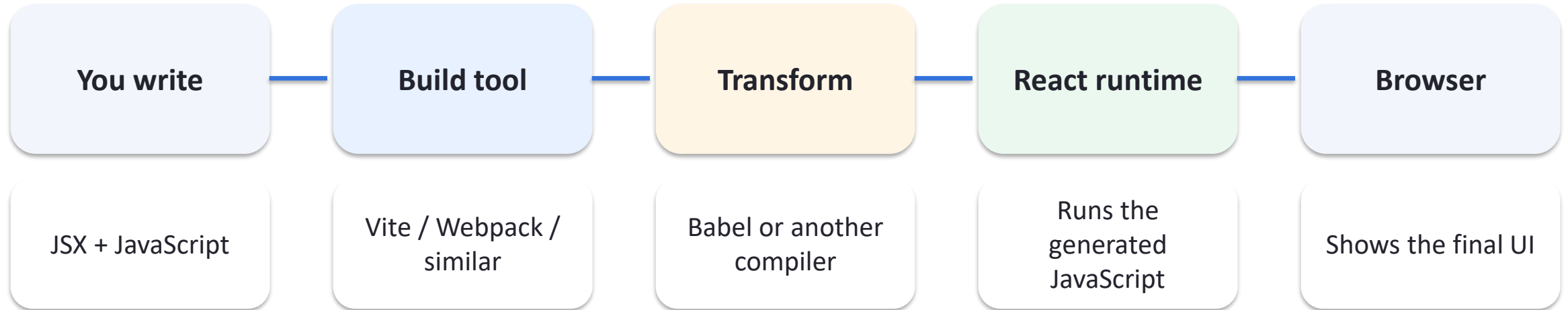


Easy way to think: Babel = translator for JavaScript code.

Important: Babel does not “run” your React app. It transforms code; the resulting JavaScript is then executed by the browser/runtime.

7. Where does Babel fit?

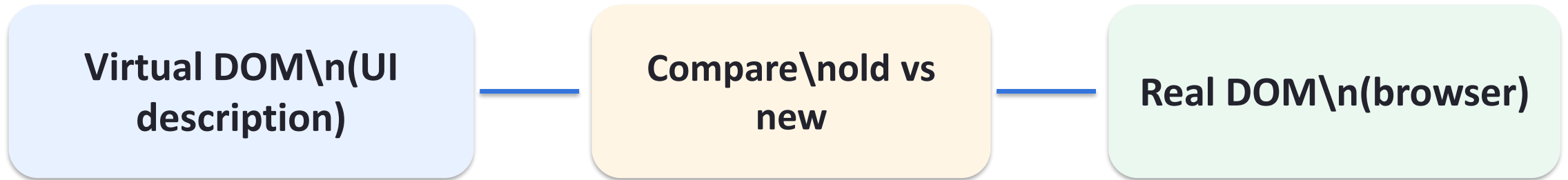
In a modern React project, several tools work together.



Note: Vite can use different transformation pipelines depending on the project setup. Students mainly need to understand the roles, not memorize every tool.

8. Virtual DOM vs Real DOM

A simple analogy: React plans the change, then updates the browser DOM.



Example: a counter changes from 0 → 1. React can update the text that needs changing instead of rebuilding the whole page.

The Virtual DOM is not the browser DOM. It is a JavaScript representation used by React to reason about UI changes.

9. Three React ideas to connect

JSX is only one piece of the React picture.

**Component\nReusable
UI**

**Props\nData from
parent**

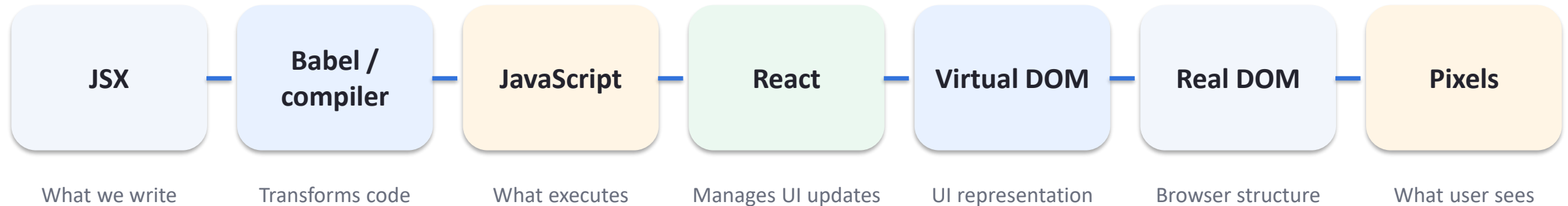
**State\nData that can
change**

**State changes → React renders the component again → a new UI description is produced
→ React updates the DOM.**

```
const [count, setCount] = useState(0);
```

10. The complete React picture

If you understand this flow, the rest becomes much easier.



Student rule of thumb: JSX is syntax • Babel/compiler is the translator • React is the UI library
• Virtual DOM is React's UI representation • DOM is the browser's actual document.